

Analyser — Developer Guide

Version: 1.8 · **Last updated:** June 2026

This guide covers the technical internals of the Analyser application for contributors and maintainers. It focuses on areas not already covered by inline code comments.

Table of Contents

1. Project Setup
 2. Architecture Overview
 - 2.1 ECharts Lazy-Loading
 3. Cross-Device Label Sync
 - 3.1 Design Goals
 - 3.2 Data Model in Redis
 - 3.3 Identity Lifecycle
 - 3.4 Sync Store (`sync.ts`)
 - 3.4a Retry / Backoff (`fetchWithRetry`)
 - 3.4b Sidebar Sync Indicator
 - 3.5 DeviceLabels Integration
 - 3.6 Toast Notification Store
 - 3.7 API Routes
 - 3.8 SyncPanel Component
 - 3.9 Layout Wiring
 - 3.10 Validation
 - 3.11 Environment Variables
 - 3.12 Local Development Setup
 - 3a. Map Tab — Metric Strip Chart
 - 3b. Data Export
 - 3c. Chart Stats Row
 - 3d. Activity Alignment — GPS Anchor
 - 3e. Data Anomaly Detection
 - 3f. Athlete Profile
 4. Responsive Layout
 5. FIT Parsing
 6. Testing
-

1. Project Setup


```

| | | # automatic ECharts-side downsampling on large datasets (>200
| | | # - TimeSeriesChart.utils.ts: computeSeriesStats(), formatStats
| | | # - StripChart.svelte: metric strip chart wrapper (map tab)
| | | # - StripToggle.svelte: line/gradient pill toggle
| | | # - StripChart.utils.ts: shouldShowGradient(), GRADIENT_COLOUR_
| | | # - png-btn.css: shared PNG download button styles
| | └─ map/ # Leaflet map + ActivityMap.component.test.ts (jsdom)
| | | # - ActivityMap.utils.ts: extractGpsPoints, metricValuesForGpsP
| | | # smoothedValues) – maps downsampled GPS points to metric val
| | | # - DeviceToggleBar.utils.ts: buildAnomalyCounts(activities) →
| | | # aggregates event-level anomaly counts across all activities
| | └─ ui/ # Layout components, controls, SyncPanel
| └─ server/
| | └─ redis.ts # Upstash Redis singleton (server-only)
| └─ stores/
| | └─ athleteProfile.ts # Athlete profile store + localStorage persistence (see
| | | # - athleteProfile: writable<AthleteProfile>({})
| | | # - setProfileField(key, value): updates store + persist
| | | # - initAthleteProfile(): loads from localStorage; SSR-
| | | # - Storage key: 'analyser-athlete-profile'; quota error
| | └─ deviceLabels.ts # Device label persistence (localStorage)
| | └─ sync.ts # Cross-device sync logic
| | └─ session.ts # Activity session state; activityColourMap assigns stat
| | └─ toast.ts # Transient toast notification store (addToast, removeTo
| | └─ viewport.ts # Responsive breakpoint store
| └─ utils/ # Pure utility functions
| | # - binarySearch.ts: lowerBound<T>(arr, key, target) – shared 1
| | # - channels.ts: deriveAvailableChannels()
| | # - deviceChannels.ts: deviceKey(), deriveDeviceLabel(), groups
| | # - fetchWithRetry.ts: fetchWithRetry(), isRetryable(), compute
| | # - formatAge.ts: formatAge(ts) → human-readable relative time
| | # - formatting.ts: formatPace(decimalMinutes) → "M:SS" string -
| | # - indoorWarnings.svelte.ts: createIndoorWarnings() composable
| | # - lapMarkers.ts: buildLapMarkers()
| | # - segments.ts: buildSegments()
| └─ validation.ts # Shared regex constants (UUID, short code)
| └─ types.ts # Domain types
| # - Anomaly, AnomalyType, DetectionStrategy, AnomalyDetectionOp
| # - ChannelKey now includes 'position' (GPS-only; no numeric se
| # - GpsPointWithDistance: lat, lon, distance, recordIndex (0(1)
| # - GpsPointWithMetric extends GpsPointWithDistance: metricValu
└─ routes/
  └─ api/
    └─ labels/
      └─ [uuid]/+server.ts # GET / PUT label store
      └─ resolve/[code]/+server.ts # GET short-code resolver
  └─ export-btn.css # Shared CSS for the Export Data button in tab bars
  └─ map-panel.css # Shared CSS for Map tab layout (.map-wrap--has-strip, .stri

```

```

└─ +layout.svelte    # App shell, initSync, ?sync= param
└─ +page.svelte      # Landing / drop zone
└─ compare/+page.svelte
└─ event/+page.svelte

```

2.1 ECharts Lazy-Loading

ECharts (≈1.1 MB / 374 kB gzipped) is **not in the initial bundle**. It is deferred to a separate async chunk and fetched only when the first chart component mounts.

How it works

All four chart components (`TimeSeriesChart` , `DeltaChart` , `MeanMaxChart` , `SegmentChart`) import ECharts through a shared singleton loader:

```

// src/lib/components/charts/echarts-loader.ts
import type * as EChartsNamespace from 'echarts';
export type EChartsModule = typeof EChartsNamespace;

let cached: Promise<EChartsModule> | null = null;

export function loadECharts(): Promise<EChartsModule> {
  if (!cached) {
    cached = import('echarts') as Promise<EChartsModule>;
  }
  return cached;
}

```

Each chart component calls `loadECharts()` inside an `async onMount`:

```

onMount(async () => {
  ec = await loadECharts(); // dynamic import (first call fetches; subs
  chart = ec.init(container, undefined, { renderer: 'canvas' });
  chart.setOption(buildOption(), { notMerge: true }); // explicit initial render
  // ... event binding, ResizeObserver, etc.
  ready = true;
});

```

Why `chart.setOption()` is called explicitly in `onMount`

Three of the four chart components (`TimeSeriesChart` , `DeltaChart` , `SegmentChart`) declare `chart` as a plain `let` variable rather than `$state`. Because `chart` is not reactive, the Svelte `$effect` that normally calls `chart?.setOption()` fires *before* the `await loadECharts()` resolves — hitting a no-op. The explicit `setOption` call immediately after `ec.init()` guarantees the first render regardless of reactive timing.

`MeanMaxChart` uses `let chart = $state<ECharts | undefined>(undefined)` — assigning the chart instance triggers the `$effect` reactively — but still includes the explicit `setOption` call for consistency and to make all four components follow the same pattern.

Loading skeleton

While `loadECharts()` is in flight the chart canvas (`bind:this={container}`) stays in the DOM with `visibility: hidden` so `ec.init()` can be called on an element with real dimensions. A shimmer placeholder div replaces the visual space:

```
{#if !ready}
  <div class="chart-canvas chart-skeleton" aria-hidden="true"></div>
{/if}
<div bind:this={container} class="chart-canvas" style:visibility={ready ? 'visible' :
```

Skeleton styles live in `src/lib/components/charts/chart-skeleton.css` (imported by all four chart components) and use CSS custom properties defined in `src/routes/layout.css`:

```
/* dark theme (default) */
--skeleton-from: #1e293b;
--skeleton-to:   #334155;

/* light theme */
--skeleton-from: #e2e8f0;
--skeleton-to:   #f1f5f9;
```

Bundle verification

`rollup-plugin-visualizer` is configured in `vite.config.ts`. Every `npm run build` writes `stats.html` to the project root (gitignored). Open it to verify ECharts appears only as a dynamic entry (`isDynamicEntry: true` in the Vite manifest) and is absent from all entry-point sync dependency trees.

Adding a new chart component

If you add a fifth chart component:

1. Import from the shared loader: `import { loadECharts, type EChartsModule } from './echarts-loader'`
2. Keep `import type { ECharts, EChartsOption } from 'echarts'` — type-only imports are erased at compile time and do not create a bundle dependency
3. Use `async onMount`, call `loadECharts()`, call `chart.setOption(buildOption(), { notMerge: true })` immediately after `ec.init()`
4. Import `chart-skeleton.css` and add the skeleton/visibility template pattern

3. Cross-Device Label Sync

PR #80 added cross-device sync for device labels. Users rename sensor pills (e.g. "Device 1" → "Polar H10") and those names are shared across all their browsers and devices without requiring an account.

3.1 Design Goals

- **Account-free** — identity is a randomly generated UUID stored in `localStorage`, not tied to any login.
- **Cloud-authoritative** — the Upstash Redis store is treated as the source of truth; local state is overwritten on pull.
- **Fire-and-forget pushes** — label changes auto-push in the background; errors surface in the UI but never block local usage.
- **Human-readable codes** — a short 8-character code (e.g. `E6Y-NXEMF`) derived from the UUID lets users type the code on another device instead of scanning a QR.
- **90-day TTL** — entries in Redis expire after 90 days of inactivity with no explicit deletion needed.

3.2 Data Model in Redis

Two key types are stored:

Key pattern	Value	TTL
<code>labels:{uuid}</code>	JSON object: <code>{ [antDeviceKey]: labelString }</code>	90 days, rolling
<code>code:{shortCode}</code>	UUID string	90 days, rolling

Both keys are refreshed together on every PUT, so activity on one extends the other's TTL automatically.

The `antDeviceKey` format is defined by `deviceKey()` in `src/lib/utils/deviceChannels.ts` — it encodes manufacturer, product ID, and device type into a stable string used as the label map key.

3.3 Identity Lifecycle

First visit:

```
crypto.randomUUID() → uuid
deriveShortCode(uuid) → shortCode
localStorage: SYNC_ID_KEY, SYNC_CODE_KEY
PUT /api/labels/{uuid} → seed remote with current labels
```

Returning visit:

```
Read uuid / shortCode from localStorage
GET /api/labels/{uuid} → pull remote labels → replaceAllLabels()
```

Label change (any visit):

```
setOnLabelChange hook → pushLabels(uuid, shortCode) [fire and forget]
```

Adopt external identity (QR / copy-link / short code):

```
adoptSyncIdentity(uuid) → store uuid in localStorage → pullLabels(uuid)
```

Reset identity:

```
crypto.randomUUID() → new uuid
```

```
localStorage cleared of old identity  
pushLabels(newUuid, newShortCode) → seed remote under new identity
```

3.4 Sync Store (`sync.ts`)

Location: `src/lib/stores/sync.ts`

Exports:

Export	Type	Description
<code>syncStatus</code>	<code>Writable<SyncStatus></code>	Reactive store for UI state (uuid, shortCode, lastSynced, error, syncing)
<code>initSync()</code>	<code>async () => (() => void) undefined</code>	Idempotent — safe to call multiple times; second call is a no-op. Returns a cleanup function on first call (call in <code>onDestroy</code> to deregister hook and allow HMR reinit); returns <code>undefined</code> if already initialised. SSR-safe.
<code>pushLabels(uuid, shortCode)</code>	<code>async</code>	Upload current labels to Redis. Retries up to 3× on transient errors. Sets <code>syncStatus.syncing</code> while in progress. Errors update <code>syncStatus.error</code> .
<code>pullLabels(uuid)</code>	<code>async</code>	Download labels from Redis. Retries up to 3× on transient errors. Sets <code>syncStatus.syncing</code> while in progress.
<code>resolveCode(code)</code>	<code>async → uuid</code>	Resolve short code to UUID via API. Retries up to 3× on transient errors. Throws on 404.
<code>adoptSyncIdentity(uuid)</code>	<code>async</code>	Replace local identity with external UUID and pull.
<code>resetSyncIdentity()</code>	<code>async</code>	Generate fresh UUID, push current labels under it.
<code>deriveShortCode(uuid)</code>	<code>string</code>	Pure function: BigInt base-36 encode of UUID, 8 chars, <code>XXX-XXXXX</code> .
<code>getSyncStatus()</code>	<code>SyncStatus</code>	Non-reactive snapshot (for use outside Svelte components).

SyncStatus shape:

```
type SyncStatus = {  
  uuid:      string | null;  
  // ...  
}
```

```

shortCode: string | null;
lastSynced: string | null; // ISO timestamp
error:      string | null;
syncing:    boolean;        // true while any sync operation is in flight (including
};

```

3.4a Retry / Backoff (`fetchWithRetry`)

Location: `src/lib/utils/fetchWithRetry.ts`

All three sync network calls (`pushLabels` , `pullLabels` , `resolveCode`) use a shared retry utility rather than bare `fetch` :

```
fetchWithRetry(input, init?, opts?): Promise<Response>
```

Default options: `maxRetries: 3` , `baseDelayMs: 1000` , `jitter: 0.25` (±25%).

Retry classification (`isRetryable`):

Response / Error	Retried?
Network error (<code>TypeError</code>)	☐ Yes
5xx	☐ Yes
429	☐ Yes
4xx (except 429)	✕ No
2xx / 3xx	✕ No

Backoff schedule (zero jitter, for clarity):

Attempt	Delay
0 → 1	~1 s
1 → 2	~2 s
2 → 3	~4 s

When the server returns a `429` with a `Retry-After` header, that value (in seconds) overrides the exponential delay for that attempt, preventing wasted retries within the rate-limit window.

After all retries are exhausted: the last `Response` is returned for HTTP errors; the last network error is thrown.

Test isolation: `_setSleepFn(fn)` replaces the internal sleep function so that retry tests do not incur real delays (used in `sync.test.ts` via `beforeEach`).

3.4b Sidebar Sync Indicator

Location: `src/lib/components/Analytics/SidebarSyncIndicator.tsx`

A persistent status indicator is always visible in the sidebar footer, showing the current sync state without requiring the user to expand the SyncPanel. It subscribes to `$syncStatus` via three `$derived.by()` values:

State	Condition	Icon	Colour
<code>syncing</code>	<code>\$syncStatus.syncing === true</code>	Animated two-arc spinner SVG	Blue (<code>#3b82f6</code>)
<code>error</code>	<code>\$syncStatus.error !== null</code>	Triangle warning SVG	Amber (<code>#fbbf24</code>)
<code>ok</code>	<code>\$syncStatus.lastSynced !== null</code>	Filled circle SVG	Green (<code>#10b981</code>)
<code>muted</code>	All others (setting up)	Filled circle SVG	<code>--color-muted</code>

The element carries `aria-live="polite"` and a dynamic `aria-label` (e.g. "Synced just now" , "Sync error: ...") for screen readers.

When `syncing === true` and `error !== null` simultaneously (i.e. auto-retry is in progress after a previous failure), `SyncPanel.svelte` shows "Retrying..." in blue instead of the normal amber "`⚠ Sync error – retry`" link.

3.5 DeviceLabels Integration

Location: `src/lib/stores/deviceLabels.ts`

Three additions support sync:

Function	Description
<code>getAllLabels()</code>	Returns a snapshot <code>Record<string, string></code> of all stored labels
<code>replaceAllLabels(map)</code>	Overwrites <code>localStorage</code> with the provided map and notifies the store
<code>setOnLabelChange(fn null)</code>	Registers a callback invoked after every <code>setDeviceLabel</code> / <code>removeDeviceLabel</code> . Pass <code>null</code> to deregister.

Only one change callback can be registered at a time. `initSync` registers its push callback after the initial push/pull, so subsequent label changes auto-push. The cleanup function returned by `initSync` calls `setOnLabelChange(null)` to deregister on teardown.

3.6 Toast Notification Store

Location: `src/lib/stores/toast.ts`

Component: `src/lib/components/ui/ToastContainer.svelte`

A lightweight, reusable toast system for surfacing transient non-blocking messages to the user. The store is framework-agnostic — any module can call `addToast` without importing Svelte components.

Export	Signature	Description
<code>toasts</code>	<code>Readable<Toast[]></code>	Reactive list of active toasts. Read-only — prevents external mutation.
<code>addToast</code>	<code>(message: string, level?: ToastLevel, duration?: number) => void</code>	Adds a toast. Default level: <code>'warning'</code> . Default duration: <code>5000 ms</code> .
<code>removeToast</code>	<code>(id: number) => void</code>	Removes a toast immediately and clears its auto-dismiss timer.

Types

```
type ToastLevel = 'info' | 'warning' | 'error';

interface Toast {
  id: number;
  message: string;
  level: ToastLevel;
}
```

Behaviour

- Each toast is assigned a unique auto-incrementing `id`.
- `addToast` schedules a `setTimeout` to call `removeToast(id)` after `duration` ms. The timer handle is stored in a `Map<id, timer>` and cleared by `removeToast` to prevent orphaned timers when a toast is dismissed early.
- `removeToast` with an unknown `id` is a safe no-op.

ToastContainer component

`ToastContainer.svelte` subscribes to `$toasts` and renders the most recent `MAX_VISIBLE` (3) toasts as a fixed-position stack (bottom-right on desktop; full-width above the bottom nav bar on phone). It is mounted **once** in `+layout.svelte` so toasts are visible on every page.

Each toast has a level-appropriate left-border accent and dismiss button (×). Colours use hardcoded semantic values (amber/blue/red) against `--color-card` / `--color-text` backgrounds, ensuring correct appearance in both light and dark themes.

Current callers

Caller	Level	Message
<code>athleteProfile.ts</code> — <code>_persist</code> catch	warning	'Athlete profile could not be saved — storage full' (QuotaExceededError) or 'Athlete profile could not be saved' (other errors)
<code>deviceLabels.ts</code> — <code>saveLabels</code> catch	warning	'Device label could not be saved — storage full' (QuotaExceededError) or 'Device label could not be saved' (other errors)
<code>DropZone.svelte</code> — <code>parse</code> error catch (single-file and multi-file)	error	'<filename>: <parser error message>' — fires alongside the existing inline error text so the error persists after navigation
<code>parser.ts</code> — <code>normalise()</code> negative elapsed	warning	'"<filename>": N record(s) with negative elapsed time were removed.'
<code>parser.ts</code> — <code>normalise()</code> out-of-order records	warning	'"<filename>": records were out of order and have been sorted automatically.'

3.7 API Routes

See the full API reference in `docs/api-reference.md`.

GET `/api/labels/[uuid]` — returns `{ labels }` or 404.

PUT `/api/labels/[uuid]` — expects `{ labels: Record<string,string>, shortCode: string }`. Writes both `labels:{uuid}` and `code:{shortCode}` to Redis with a 90-day TTL.

GET `/api/labels/resolve/[code]` — resolves short code to UUID. Rate-limited at 10 req/min/IP via Redis `INCR / EXPIRE` pipeline (globally enforced across all Vercel instances). Responds 429 with `Retry-After` header on limit exceeded; fails open if Redis is unavailable.

All routes validate inputs against `UUID_REGEX` / `SHORT_CODE_REGEX` from `src/lib/validation.ts` and return structured JSON errors on 400/404/429/500.

3.8 SyncPanel Component

Location: `src/lib/components/ui/SyncPanel.svelte`

A collapsible panel rendered in the Sidebar footer. It is always mounted (not conditionally rendered) so the sync toggle row is always visible.

Key internal state:

State	Type	Purpose
<code>isOpen</code>	<code>boolean</code>	Controls panel expansion
<code>codeInput</code>	<code>string</code>	Bound to the short-code text input
<code>codeError</code> / <code>codeSuccess</code>	<code>string null</code> / <code>boolean</code>	Inline validation feedback
<code>isResolving</code>	<code>boolean</code>	Disables input during async resolve
<code>copied</code>	<code>boolean</code>	Briefly true after clipboard write, drives button label

Derived values:

- `syncUrl` — `${origin}?sync=${uuid}`, used for both QR generation and copy-link.
- `qrSvg` — generated client-side via `uqr ( renderSVG(syncUrl) )`. No server round-trip.

The component validates code input against `SHORT_CODE_REGEX` before hitting the API, providing immediate feedback on format errors.

3.9 Layout Wiring

`src/routes/+layout.svelte`

Four wiring points in `onMount`:

1. `initTheme()` — applies the persisted theme preference.
2. `initAthleteProfile()` — loads athlete profile from `localStorage` into the `athleteProfile` store. SSR-safe; a no-op when `typeof localStorage === 'undefined'`.
3. `initSync()` — idempotent; safe if called more than once (e.g. during HMR). Stores the returned cleanup function in `cleanupSync`.
4. URL parameter handling: if the page loads with `?sync={uuid}`, `adoptSyncIdentity(uuid)` is called and the param is stripped from the URL using `history.replaceState` to avoid sharing it inadvertently via browser history.

`onDestroy` calls `cleanupSync?.()` — deregisters the label-change hook and resets the initialised flag, enabling a clean reinit if the layout remounts (HMR).

3.10 Validation

`src/lib/validation.ts`

Shared between API routes (server) and `SyncPanel` (client) to avoid duplication:

```
UUID_REGEX      = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i
SHORT_CODE_REGEX = /^[A-Z0-9]{3}-[A-Z0-9]{5}$/i
```

3.11 Environment Variables

Variable	Required	Description
UPSTASH_REDIS_REST_URL	Yes	REST endpoint for your Upstash Redis database
UPSTASH_REDIS_REST_TOKEN	Yes	Read-write token for the database

These are read via `$env/dynamic/private` (SvelteKit server-only) in `src/lib/server/redis.ts`. They must be set in:

- **Local development:** `.env.local` (git-ignored, never committed)
- **Vercel:** Project Settings → Environment Variables

The Redis singleton (`getRedis()`) throws an explicit error if credentials are absent, which the API routes catch and surface as HTTP 500. This prevents silent failures.

3.12 Local Development Setup

1. Create an Upstash account and create a Redis database (free tier is sufficient).
2. Copy the REST URL and token from the Upstash console.
3. Create `.env.local` in the project root:

```
UPSTASH_REDIS_REST_URL=https://your-db.upstash.io
UPSTASH_REDIS_REST_TOKEN=your-token-here
```

4. Run `npm run dev` — sync features are fully functional locally.

Tip: If you do not configure Redis, the app still works — sync API calls will return 500 and `syncStatus.error` will be set, but all FIT file analysis features continue to function normally.

3a. Map Tab — Metric Strip Chart

PR #82 added a co-visible metric strip chart to the Map tab on both `/compare` and `/event` pages. When the user selects a metric in the **Colour by** picker, a chart for that channel slides in below the map with full bidirectional hover sync.

Component hierarchy

```
compare/+page.svelte (or event/+page.svelte)
├── .map-panel (flex column)
│   ├── .map-wrap (flex: 7)
│   │   └── <ActivityMap
│   │       ├── onMetricChannelChange={ch => mapMetricChannel = ch}
│   │       ├── hoveredDistance={stripHoveredDistance}
│   │       └── />
│   └── .strip-wrap (flex: 3, shown only when mapMetricChannel !== null)
│       └── <CollapsiblePanel> (collapses on tablet/phone)
```

```

└─ <StripChart
    channel={mapMetricChannel}
    seriesInputs={stripSeriesInputs}
    lapMarkers={...}
    onHoverDistance={handleStripHoverDistance}
    externalHoverDistance={mapStripHoveredDistance}
  />

```

Hover sync

The strip chart uses a **second, independent hover loop** (`stripHoveredDistance` / `mapStripHoveredDistance`), completely separate from the existing Charts-tab ↔ Map loop (`chartHoveredDistance` / `mapHoveredDistance`). This separation is intentional:

- The existing loop is **gated on** `$xAxisMode === 'distance'` — it only fires when the Charts tab is in distance mode.
- The strip loop is **never gated** — the strip chart always operates in distance mode (`forceDistanceAxis={true}`), so the sync is always valid regardless of the global x-axis setting.

Key design points

Concern	Approach
Strip always uses distance axis	<code>forceDistanceAxis={true}</code> prop on <code>TimeSeriesChart</code> inside <code>StripChart</code>
Isolated zoom/pan	<code>groupId="map-strip"</code> — separate ECharts connect group from <code>"compare-charts"</code> / <code>"event-charts"</code>
Gradient mode	<code>shouldShowGradient()</code> utility in <code>StripChart.utils.ts</code> ; single-file only (multi-file stays as lines)
Shared layout CSS	<code>.map-wrap--has-strip</code> and <code>.strip-wrap</code> styles extracted to <code>src/routes/map-panel.css</code> and imported in both pages
Magic-string safety	Gradient sentinel value is <code>GRADIENT_COLOUR_TOKEN = '__gradient__'</code> (exported constant, not inline string)

`ActivityMap` — `onMetricChannelChange` callback

`ActivityMap.svelte` accepts an optional `onMetricChannelChange?: (channel: ChannelKey | null) => void` prop. It fires via a `$effect` whenever the internal `metricChannel` state changes (on picker selection and on file change/reset). Existing callers that do not pass this prop are unaffected.

3b. Data Export

PR #83 added Excel workbook and per-chart PNG downloads. PR #111 added CSV export and extracted shared column utilities into `columns.ts`.

Module layout

File	Responsibility
<code>src/lib/export/columns.ts</code>	Shared column utilities used by both CSV and Excel builders: <code>CHANNEL_KEYS</code> , <code>presentChannels()</code> , <code>buildHeaderLabel()</code> , <code>formatCellValue()</code> , <code>escapeCsvField()</code> ; <code>ExportFormat</code> type (<code>'csv'</code> <code>'xlsx'</code>)
<code>src/lib/export/csv.ts</code>	<code>buildCsv(activities)</code> — builds an RFC 4180 CSV string in memory
<code>src/lib/export/excel.ts</code>	<code>buildWorkbook(activities)</code> — builds a SheetJS workbook in memory and returns it as an <code>ArrayBuffer</code>
<code>src/lib/export/exportActivities.ts</code>	<code>exportActivities(activities, format)</code> — async entry point: lazy-imports <code>csv.ts</code> or <code>excel.ts</code> based on <code>format</code> (defaults to <code>'csv'</code>), then calls <code>triggerDownload</code>
<code>src/lib/export/download.ts</code>	<code>triggerDownload(data, filename, mime)</code> , <code>downloadPng(dataUrl, filename)</code> , <code>localDateString()</code>

Both `csv.ts` and `excel.ts` are lazy-imported in `exportActivities.ts` so that SheetJS and the CSV builder are only bundled into dynamically loaded chunks, keeping the main bundle lighter.

Shared column utilities (``columns.ts``)

`columns.ts` is the single source of truth for which channels appear in exports and how values are formatted. Both `csv.ts` and `excel.ts` import from it to prevent drift as new channels are added.

- `CHANNEL_KEYS` — ordered list of all exportable `ChannelKey` values
- `presentChannels(records)` — filters `CHANNEL_KEYS` to those with at least one non-null value across the provided records
- `buildHeaderLabel(key)` — returns the human-readable column header (e.g. `"Heart Rate (bpm)"`)
- `formatCellValue(key, value)` — formats a raw value for output: `pace` → `"M:SS"` string; timestamps → `Date` ; others → number or `null`
- `escapeCsvField(value)` — wraps a string in double-quotes and doubles any internal quotes per RFC 4180; returns the original string if no quoting is needed

CSV structure (``csv.ts``)

`buildCsv(activities)` produces an RFC 4180 string:

- **Single activity** — header row + one row per `ActivityRecord` ; no `activity` column
- **Multiple activities** — header row + all records from all activities in order; `activity` column (source filename) prepended as column 1

- Channels with all-null values across all records are omitted
- Pace formatted as "M:SS" ; timestamps as ISO 8601 strings
- Lines separated by CRLF; file ends with a trailing CRLF

Excel workbook structure (`excel.ts`)

`buildWorkbook` produces:

1. **Summary sheet** — one row per activity with filename, sport, start time (Excel datetime), distance (km), and elapsed time (H:MM:SS or M:SS).
2. **Per-activity sheets** — one row per `ActivityRecord`. Sheet names are taken from `activity.filename`, truncated to 31 characters (Excel limit), and de-duplicated with (N) suffixes if needed.

Column selection is dynamic: `presentChannels()` from `columns.ts` filters to channels with at least one non-null value in the activity.

Pace is formatted as an "M:SS" string by `formatCellValue()` rather than stored as a decimal, since Excel has no built-in pace format and decimal values are not human-readable.

Timestamps use JavaScript `Date` objects with `cellDates: true` in the SheetJS write options, with `yyyy-mm-dd hh:mm:ss` cell format applied to datetime columns.

PNG download

Each chart component calls `chart.getDataURL({ type: 'png', pixelRatio: 2, backgroundColor: bgColour })` on its ECharts instance, then passes the resulting base64 data URL to `downloadPng()`. `downloadPng` decodes the base64 string to a `Uint8Array`, wraps it in a `Blob`, and calls `triggerDownload` to initiate the browser download.

The background colour is read from the CSS custom property `--color-bg-primary` at download time so that the PNG matches the user's current theme.

`localDateString`

`new Date().toISOString()` returns UTC, which rolls over to the next day before midnight for users in UTC—timezones. `localDateString()` uses `toLocaleDateString('en-CA')` instead, which produces an ISO-format (`YYYY-MM-DD`) date in the user's local timezone.

3c. Chart Stats Row

PR #85 added a compact stats row below each `TimeSeriesChart` showing minimum, average, and maximum for every active series (added in PR #86). Stats update reactively when devices are toggled, and recalculate when the user zooms into a region.

Pace inversion (PR #88): Because pace is an inverted metric (lower min/km = faster), `computeSeriesStats()` swaps the min/max values for the `pace` channel so that `SeriesStats.max` holds the fastest (numerically smallest) pace and `SeriesStats.min` holds the slowest. The template renders pace as `max / avg / min` so numbers still read ascending left-to-right, consistent with the pace chart's inverted y-axis.

Module layout

File	Responsibility
<code>src/lib/components/charts/TimeSeriesChart.utils.ts</code>	<code>computeSeriesStats()</code> , <code>formatStatValue()</code> , <code>SeriesStats</code> interface
<code>src/lib/components/charts/TimeSeriesChart.svelte</code>	<code>zoomRange</code> state, <code>seriesStats</code> derived value, <code>.chart-stats</code> HTML/CSS

Key functions

`formatStatValue(value, channel)` — formats a raw number for display. Pace channels use `paceFormat()` (M:SS — re-exported from `$lib/utils/formatting` as the shared `formatPace`); integer channels (heartRate, power, cadence) round to zero decimal places; float channels (speed, temperature) show one decimal.

`computeSeriesStats(data, channel, label, colour, xRange?)` — slices `data` to the optional `xRange` window, passes y-values to `summarise()`, and returns a `SeriesStats` object (or `null` if no data). The `xRange` is provided by the zoom handler.

`SeriesStats` interface:

```
interface SeriesStats {
  label: string;
  colour: string;
  min: string; // formatted for display (pace: slowest; others: numeric minimum)
  avg: string; // formatted for display
  max: string; // formatted for display (pace: fastest; others: numeric maximum)
  minRaw: number;
  avgRaw: number;
  maxRaw: number;
  count: number;
  xMin: number;
  xMax: number;
  unit: string;
}
```

Zoom integration

`TimeSeriesChart.svelte` listens to ECharts' `dataZoom` event. When a zoom is active, the x-axis extent is read from ECharts' internal model via `chart.getModel().getComponent('xAxis', 0)` and stored in `zoomRange`. The `seriesStats` derived value re-runs whenever `zoomRange` changes, slicing each series' data to the visible window.

`zoomRange` is reset to `undefined` on every chart rebuild (new data, smoothing change, axis mode change), so stats always reflect the full dataset after a rebuild.

Data source vs Summary tab

The stats row and the Summary tab use the same `summarise()` function but different data sources:

Surface	Data source	Notes
Stats row	<code>buildData()</code>	Smoothed; distance-axis interpolated when in distance mode
Summary tab	<code>extractChannel(activity.records, ch)</code>	Raw unsmoothed records

When smoothing is 1 s and the axis is time, the two surfaces agree closely. In distance mode or with smoothing > 1 s, values may differ. Both surfaces carry a `title` tooltip explaining the difference.

3d. Activity Alignment — GPS Anchor and Indoor Detection

PR #91 replaced raw start-time-difference alignment with GPS-anchor-based alignment. PR #92 extended this to handle indoor activities (TrainerRoad, Zwift, treadmill) where GPS is absent or synthetic.

Indoor activity detection

`classifyIndoor(subSport, records)` in `src/lib/fit/parser.ts` determines `activity.isIndoor` at parse time using two signals:

1. **sub_sport field** (primary) — if the FIT session contains a recognised indoor `sub_sport` value, the activity is classified as indoor regardless of GPS presence. The full set is:

```
const INDOOR_SUB_SPORTS = new Set([
  'indoor_cycling', 'virtual_activity', 'spin', 'stationary_bike',
  'treadmill', 'indoor_rowing', 'indoor_running', 'indoor_walking',
  'virtual_ride', 'virtual_run',
]);
```

Key insight: Both Zwift (`virtual_activity`) and TrainerRoad (`indoor_cycling`) embed fake GPS coordinates in every record (Zwift uses virtual-world positions; TrainerRoad uses a Central Park, NYC placeholder). GPS presence cannot be used as the indoor detector — `sub_sport` is authoritative.

2. **GPS-absence fallback** — only fires when `sub_sport` is entirely absent (`undefined`) and all records lack a GPS position. Covers older devices that do not write `sub_sport` .

`activity.subSport` stores the raw `sub_sport` string from the FIT session (or `undefined`).
`activity.isIndoor: boolean` is always populated.

Anchor hierarchy

`findAnchor(activity)` in `src/lib/align/anchor.ts` branches on `activity.isIndoor`:

Outdoor path (unchanged from #91):

Priority	Source	AnchorSource
1	FIT timer-start event (within 30 s)	'timer'
2	First GPS record with speed > 0	'gpsMovement'
3	First GPS record (stationary)	'gpsFix'
4	File start	'fileStart'

Indoor path (added in #92 — GPS signals are skipped entirely):

Priority	Source	AnchorSource
1	FIT timer-start event (within 30 s)	'timer'
2	First <code>workout_step</code> timestamp (within 30 s)	'workoutStep'
3	First record with speed/power/cadence > 0	'indoorMovement'
4	File start	'fileStart'

The result is an `AlignmentAnchor` stored on `Activity` at parse time:

```
type AnchorSource = 'timer' | 'gpsMovement' | 'gpsFix' | 'fileStart'
                    | 'workoutStep' | 'indoorMovement';

interface AlignmentAnchor {
  recordIndex: number;
  distanceMetres: number;
  elapsedSeconds: number;
  timestamp: Date;
  source: AnchorSource;
}
```

`activity.anchor` is computed once in `parser.ts → normalise()` and available to all downstream code. The new `Activity` fields supporting indoor alignment are:

Field	Type	Description
subSport	string undefined	Raw FIT sub_sport value
isIndoor	boolean	Computed by classifyIndoor() at parse time
firstIndoorMovementIndex	number null	First record with speed/power/cadence > 0
firstWorkoutStepTime	Date null	Timestamp of first workout_step FIT message

Time-axis alignment

computeAnchoredOffsets(activities) in src/lib/align/timestamp.ts aligns files relative to the first loaded file's anchor timestamp rather than raw startTime values.

The TimeOffsetControl component shows a colour-coded anchor badge per file. Labels and tooltips are defined in TimeOffsetControl.utils.ts as ANCHOR_LABELS and ANCHOR_TITLES (both typed as Record<AnchorSource, string> for compile-time completeness):

Badge	Colour	Anchor source
Timer	Blue	'timer'
GPS move	Blue	'gpsMovement'
GPS fix	Amber	'gpsFix'
File start	Grey	'fileStart'
Workout	Green	'workoutStep'
Indoor	Green	'indoorMovement'

Distance-axis re-zeroing

interpolateToDistanceAxis(records, axis, distanceOffset) shifts the distance axis so that distanceOffset maps to 0 km. activity.anchor.distanceMetres is passed as distanceOffset in both Compare and Event pages.

Indoor/mixed session warnings — `createIndoorWarnings()`

src/lib/utils/indoorWarnings.svelte.ts exports a Svelte 5 runes composable that encapsulates the shared indoor warning state used by both Compare and Event pages:

```
const indoor = createIndoorWarnings(() => $activities);
// indoor.hasMixedIndoorOutdoor – any indoor + any outdoor file loaded (requires ≥2 files)
// indoor.allIndoor – all loaded files are indoor (single file or more)
// indoor.mixedWarningDismissed – dismissal state for amber banner
// indoor.indoorInfoDismissed – dismissal state for blue banner
```

The composable owns the dismissal `reset $effect` (fires on every `$activities` change). Each page adds two separate `$effect` blocks for axis auto-switching:

- **Mixed session:** `if (hasMixedIndoorOutdoor && $xAxisMode === 'distance') xAxisMode.set('time')` — locks to Time because mixed distance axes are incompatible.
- **All-indoor session:** `if (allIndoor && untrack(() => $xAxisMode) === 'distance') xAxisMode.set('time')` — defaults to Time on file load but uses `untrack()` so the user can manually switch back to Distance. `untrack()` prevents `$xAxisMode` from becoming a reactive dependency, limiting the effect to firing only when the file set changes.

CSS for the banners is in `src/routes/indoor-warning.css` (shared by both pages).

Proximity warning

`anchorsAreDistant(activities)` returns `true` when any two GPS-anchored activities have anchor positions more than `GPS_PROXIMITY_THRESHOLD_M` (50 m) apart. Both pages show a dismissible amber banner. Indoor activities whose anchors have no GPS position are excluded from the comparison, so a pure-TR file (no GPS) paired with a Zwift file never triggers a false proximity warning.

3e. Data Anomaly Detection

PR #147 added a data quality layer that detects sensor anomalies at parse time and surfaces them in the UI.

Detection engine — ``src/lib/analytics/anomalies.ts``

`detectAnomalies(records, options?)` is called once in `normalise()` and returns a sorted `Anomaly[]` stored on `Activity.anomalies`. Detection is $O(n)$ per channel.

Types and strategies:

```
type AnomalyType      = 'spike' | 'dropout' | 'gps-drift';
type DetectionStrategy = 'threshold-relative' | 'statistical';

interface Anomaly {
  channel:      ChannelKey;
  recordIndex:  number;
  type:         AnomalyType;
  value:        number;      // raw sensor value at that record
  detectionStrategy: DetectionStrategy;
}
```

Spike detection — channels: `heartRate`, `power`, `cadence`

Condition	Threshold	Strategy
HR + <code>options.athleteProfile.maxHR</code> set	<code>maxHR + 10 bpm</code>	<code>threshold-relative</code>
Power + <code>options.powerSource === 'stryd' + cp</code> set	<code>cp × 1.5</code>	<code>threshold-relative</code>
Fallback (no profile)	<code>mean + 4σ</code>	<code>statistical</code>

A run above threshold must last ≤ 3 s to be flagged as a spike (longer sustained high values are real effort, not glitches).

Dropout detection — channels: `heartRate`, `power`, `cadence`

A channel value of `0` or `null` for more than 10 consecutive seconds mid-activity (excluding the first/last 30 s warmup/cooldown) is flagged as `type: 'dropout'`, `detectionStrategy: 'threshold-relative'`.

GPS drift detection — channel: `position`

Consecutive GPS positions are compared using the Haversine formula. If the implied speed between two points exceeds $3\times$ the recorded device speed, the record is flagged as `type: 'gps-drift'`, `detectionStrategy: 'threshold-relative'`. GPS drift anomalies use the `'position'` ChannelKey — a GPS-only channel with no numeric chart series. They surface only through the DeviceToggleBar GPS section and as red circle markers on the map.

Options pattern — `AnomalyDetectionOptions`:

```
interface AnomalyDetectionOptions {
  powerSource?: 'stryd' | 'native';
  athleteProfile?: AthleteProfile; // from $lib/types — weight, ftp, cp, maxHrCycling
}
```

Currently `detectAnomalies(records)` is called without options from `parser.ts` (statistical fallback for all channels). When issue #117 (power source detection) ships, only the call site changes — the detection logic itself is unchanged. The `athleteProfile` option is wired and ready; `maxHrCycling` and `maxHrRunning` are used for sport-aware spike thresholds when provided.

Event grouping — `groupAnomalyEvents()`

`detectAnomalies` returns one `Anomaly` per record. A 40-second dropout produces 40 entries. `groupAnomalyEvents(anomalies)` collapses contiguous runs of the same channel + type (consecutive `recordIndex` values differing by ≤ 1) into a single entry (the first record of each run). This converts per-record counts to per-event counts for badge display.

```
// 40 dropout records → 1 dropout event
groupAnomalyEvents(activity.anomalies.filter(a => a.channel === 'heartRate'))
// → [Anomaly { recordIndex: startOfDropout, type: 'dropout', ... }]
```

Both Compare and Event pages call `groupAnomalyEvents()` before passing anomalies to `TimeSeriesChart` (one `markPoint` per event, not per record).

UI surfaces

DeviceToggleBar badges (`src/lib/components/ui/DeviceToggleBar.svelte`)

`buildAnomalyCounts(activities)` in `DeviceToggleBar.utils.ts` aggregates event-level counts across all activities and returns a `Map<ChannelKey, AnomalyCount>`. The badge `△ N` is rendered next to each channel label that has anomalies. Tooltip breaks down counts (e.g. "2 spikes, 1 dropout"). A standalone **GPS** section appears at the bottom when `anomalyCounts.get('position')` is non-zero — GPS drift anomalies are surfaced here rather than in any numeric channel group.

ChannelToggleBar badges (`src/lib/components/ui/ChannelToggleBar.svelte`)

The Event page passes `anomalyCounts` to `ChannelToggleBar`. Each channel pill shows an inline `△N` badge when that channel has anomaly events. No GPS section (Event page has no ActivityMap drift markers).

TimeSeriesChart markPoints (`src/lib/components/charts/TimeSeriesChart.svelte`)

`TimeSeriesChart` accepts `anomalies?: Anomaly[]` (pre-filtered to the chart's channel and pre-grouped via `groupAnomalyEvents`). A red diamond `markPoint` is rendered at each anomaly's x-axis position on the **first visible series**. Positions use `anomalyXValue()` from `TimeSeriesChart.utils.ts` — which returns `elapsedSeconds` in time mode and kilometres in distance mode. Clicking a `markPoint` zooms the chart to ± 15 s (time mode) or ± 50 m (distance mode) around the anomaly.

ActivityMap drift circles (`src/lib/components/map/ActivityMap.svelte`)

`ActivityMap` reads `activity.anomalies` directly (no separate prop). GPS-drift anomalies are filtered by `a.type === 'gps-drift'`, and each anomaly's corresponding `ActivityRecord.position` is rendered as a red `L.circleMarker` (radius 5, `#ef4444`). Non-GPS anomalies are ignored on the map.

`position` ChannelKey

'position' was added to the `ChannelKey` union to give GPS drift anomalies a first-class channel assignment. It does NOT appear in `ALL_RECORD_CHANNELS` (the ordered list used by `channelsPresentInRecords`), so it is never returned by `deriveAvailableChannels()` and never rendered as a chart series. Its sole purpose is to carry GPS drift anomaly data through `buildAnomalyCounts` to the DeviceToggleBar GPS section.

3f. Athlete Profile

PR #150 added a local athlete profile system: users enter their training thresholds once and the app uses them to annotate charts and summary tables with zone context.

Type — `AthleteProfile`

Location: `src/lib/types.ts`

```
interface AthleteProfile {
  weight?:      number; // kg
  ftp?:         number; // W – cycling Functional Threshold Power
  maxHrCycling?: number; // bpm – cycling max heart rate
  cp?:          number; // W – running Critical Power (Stryd model)
  maxHrRunning?: number; // bpm – running max heart rate
  lthr?:        number; // bpm – Lactate Threshold Heart Rate (fallback)
}
```

All fields are optional. The store starts as `{}` and any field can be removed by setting it to `undefined`.

Store — `athleteProfile.ts`

Location: `src/lib/stores/athleteProfile.ts`

Export	Description
<code>athleteProfile</code>	<code>Writable<AthleteProfile></code> — reactive store
<code>setProfileField(key, value)</code>	Updates one field in the store and persists to <code>localStorage</code> . Deletes the key when <code>value</code> is <code>undefined</code> or <code>NaN</code> . Emits a <code>'warning'</code> toast on <code>QuotaExceededError</code> .
<code>initAthleteProfile()</code>	Loads persisted profile from <code>localStorage</code> . SSR-safe (<code>typeof localStorage === 'undefined'</code> guard). Called once in <code>+layout.svelte</code> <code>onMount</code> .

Storage key: `'analyser-athlete-profile'`. Stored as a JSON object (only set keys are present — unset fields are omitted).

Zone utility module — `zones.ts`

Location: `src/lib/analytics/zones.ts` (re-exported via `src/lib/analytics/index.ts`)

All functions are pure with no side effects. Every denominator function returns `0` for zero or negative input to prevent `Infinity` results.

Function	Signature	Description
<code>hrZone</code>	<code>(bpm, profile, sport) → HrZone null</code>	Sport-aware HR zone. Uses <code>maxHrCycling</code> for 'cycling', <code>maxHrRunning</code> otherwise. Falls back to the other sport's maxHR, then to LTHR. Returns <code>null</code> when no threshold is set.
<code>hrZoneFromMaxHR</code>	<code>(bpm, maxHR) → HrZone</code>	5-zone % of maxHR model (Z1 < 60%, ..., Z5 ≥ 90%)
<code>hrZoneFromLTHR</code>	<code>(bpm, lthr) → HrZone</code>	5-zone % of LTHR model (Z1 < 81%, ..., Z5 ≥ 100%)
<code>lthrToEstimatedMaxHR</code>	<code>(lthr) → number</code>	<code>lthr / 0.92</code> — Friel/Coggan approximation. Use when maxHR is unavailable.
<code>cpZone</code>	<code>(watts, cp) → CpZone</code>	Stryd 5-zone CP model (Z1 < 75%, ..., Z5 ≥ 120%)
<code>ftpPct</code>	<code>(watts, ftp) → number</code>	<code>round((watts / ftp) * 100)</code> . Returns <code>0</code> when <code>ftp ≤ 0</code> .
<code>cpPct</code>	<code>(watts, cp) → number</code>	<code>round((watts / cp) * 100)</code> . Returns <code>0</code> when <code>cp ≤ 0</code> .
<code>wPerKg</code>	<code>(watts, weightKg) → number</code>	<code>round((watts / weightKg) * 10) / 10</code> . Returns <code>0</code> when <code>weightKg ≤ 0</code> .
<code>hrZoneBoundaries</code>	<code>(maxHR) → ZoneBand[]</code>	5 contiguous <code>{ min, max, zone }</code> bands for ECharts <code>markArea</code> construction
<code>cpZoneBoundaries</code>	<code>(cp) → ZoneBand[]</code>	5 contiguous <code>{ min, max, zone }</code> bands for ECharts <code>markArea</code> construction

How the profile flows into the UI

All three consumers receive `athleteProfile` and `sport` as props from their respective page components:

```
<!-- compare/+page.svelte -->
<TimeSeriesChart athleteProfile={$athleteProfile} sport={$activities[0]?.sport ?? ''}
<MeanMaxChart    athleteProfile={$athleteProfile} sport={$activities[0]?.sport ?? ''}
```

TimeSeriesChart.svelte — adds HR zone `markArea` bands for the `heartRate` channel and CP zone bands for `power` when `sport === 'running'`. Uses the cross-sport maxHR fallback inline (same logic as `hrZone()`), then calls `lthrToEstimatedMaxHR()` for the LTHR path.

MeanMaxChart.svelte — adds an FTP (cycling) or Critical Power (running) dashed `markLine`. Adds a second w/kg Y-axis when `weight` is set and `weight > 0`.

compare/+page.svelte — `buildCellContext(ch, avg, sport, profile)` derives the Summary tab context object:

```
// For power channels:
{ pctLabel: '92% FTP' | '88% CP', wkg: '3.6' }

// For heartRate:
{ zone: 2 }

// Returns null when no relevant profile field is set
```

Sidebar SSR workaround

`AthleteProfilePanel` is wrapped in `{#if mounted}` in `Sidebar.svelte` :

```
let mounted = $state(false);
onMount(() => { mounted = true; });

{#if mounted}<AthleteProfilePanel />{/if}
```

This prevents an SSR hydration crash where the panel's `<button>` element was absent from server-rendered HTML, causing Svelte 5's `sibling()` to consume the wrong DOM node. The workaround causes the Profile toggle button to appear only after client hydration (brief FOUC on first load). Tracked in issue #151.

4. Responsive Layout

The app is fully responsive across three viewport tiers. Breakpoints are defined in two places that must stay in sync:

- **CSS:** `--bp-phone: 480px` and `--bp-tablet: 768px` in `src/routes/layout.css`
- **JS:** `PHONE_MAX = 480` and `TABLET_MAX = 768` in `src/lib/stores/viewport.ts`

The `viewport` readable store emits `'phone' | 'tablet' | 'desktop'` via `window.matchMedia` listeners. It is SSR-safe (defaults to `'desktop'`).

Key responsive components:

Component	Behaviour change at $\leq 768\text{px}$
<code>Sidebar.svelte</code>	Switches to <code>position: fixed</code> slide-out drawer
<code>+layout.svelte</code>	Renders hamburger button (fixed, top-left, 44×44px) and backdrop
<code>CollapsiblePanel.svelte</code>	Wraps toolbar controls; <code>display: contents</code> on desktop, collapsible on mobile
<code>+layout.svelte</code> (phone only)	Renders bottom navigation bar at $\leq 480\text{px}$

5. FIT Parsing

FIT files are parsed by `fit-file-parser` running in a **Web Worker** to avoid blocking the UI thread. The worker is created in `src/lib/fit/parser.ts`.

Key parsing details:

- `STRING_DEVICE_TYPE` mapping converts `fit-file-parser`'s snake_case string outputs (e.g. `'heart_rate'`) back to numeric ANT+ constants.
- Device deduplication: duplicate `device_index` entries are discarded (only first occurrence retained).
- Two-pass channel allocation: external sensors (known ANT+ types) claim channels first; the watch claims remaining channels; any still-unclaimed channels are merged into the first device as a safety net.
- Enhanced fields: `enhanced_speed` and `enhanced_altitude` are preferred over `speed` and `altitude`.
- Stryd developer fields: `Power` (capital P), `stance_time`, `step_length` are mapped to standard channel keys.

Sport-specific post-processing is applied after record normalisation:

Sport	Function	Reason
<code>running</code>	<code>applyRunningCadenceDoubling(records)</code>	FIT cadence is single-leg (one foot per minute). Doubles to get conventional spm (steps per minute).
<code>cycling</code>	<code>removeCyclingPace(records)</code>	Pace (min/km) is not a meaningful cycling metric. Clears all <code>pace</code> values so the channel is excluded from device streams and charts entirely.

Both functions are exported from `parser.ts` for unit testing.

Record validation is applied in `normalise()` after record normalisation and before sport-specific post-processing (PR #142):

Function	Behaviour	Toast / log
<code>filterNegativeElapsed(records)</code>	Returns a new array with all records where <code>elapsedSeconds < 0</code> removed. Broken device clocks can emit these.	<code>console.warn</code> + <code>'warning'</code> toast if any are removed.
<code>ensureSortedByElapsed(records)</code>	Sorts <code>records</code> in-place by <code>elapsedSeconds</code> if out of order; returns <code>true</code> when sorting was required.	Caller (<code>normalise()</code>) emits <code>console.warn</code> + <code>'warning'</code> toast on <code>true</code> .

Both functions are exported from `parser.ts` for unit testing. Well-formed files pass through both with no output.

Lap building is handled by `buildLaps(fitLaps, records)`, which is also exported for direct unit testing. It derives `startDistance` and `endDistance` from the running cursor position in the records array rather than from the FIT lap's `start_distance` field. This makes it resilient to Garmin devices that write `start_distance = 0` for every lap (per-lap relative) rather than cumulative from the session start. Laps with `total_distance = 0` are returned as zero-length entries (`startIndex === endIndex`, `startDistance === endDistance`) without advancing the cursor.

The cursor advancement uses `records[cursor].distance <= targetDist + DISTANCE_EPSILON_M` (where `DISTANCE_EPSILON_M = 0.5`) to absorb GPS floating-point drift at lap boundaries. The constant is exported for use in tests and any future callers that need the same tolerance.

Anomaly detection is called at the end of `normalise()` after sport-specific post-processing and before `buildLaps()`. See section 3e for details.

Available channel pre-computation — `channelsPresentInRecords(records)` scans each `ActivityRecord` once across all 14 channel keys and returns a `Set<ChannelKey>` of those with at least one non-null value. It is called once inside `normalise()` and the result is stored on `Activity` as `availableChannels: Set<ChannelKey>` (PR #108).

`deriveAvailableChannels(activities)` in `src/lib/utils/channels.ts` unions those pre-computed sets rather than re-scanning records on every render call. It returns the channel list in the canonical ordering defined by `ALL_RECORD_CHANNELS` (exported from `parser.ts`), which also replaces the formerly duplicated `ALL_CHANNELS` constant that used to live only in `channels.ts`.

6. Testing

Unit tests use **Vitest**. Test files live alongside the modules they test (e.g. `TimeSeriesChart.test.ts` next to `TimeSeriesChart.svelte`).

```
npm test           # run all tests
npm run test:watch # watch mode
```

Test environments

Most tests run in the default `node` environment (pure utility functions with no DOM). Component tests that mount Svelte components require a DOM and use `jsdom`.

To mark a test file as a component test, add this directive as the **first line** of the file:

```
// @vitest-environment jsdom
```

Why the `browser` resolve condition is needed for component tests:

Svelte 5 exports both a browser build (`index.js`) and a server build (`index-server.js`). In `node` environment, Vite resolves to the server build, which throws `"mount(...) is not available on the server"` . The `vite.config.ts` sets `resolve.conditions: ['browser']` **only when** `mode === 'test'` so that jsdom component tests use the browser build. The production build is unaffected (Vite's built-in defaults include `browser` for client bundles).

```
// vite.config.ts – mode-conditional resolve
export default defineConfig(({ mode }) => ({
  plugins: [tailwindcss(), sveltekit(), visualizer({ filename: 'stats.html', open: false }),
  build: { chunkSizeWarningLimit: 1200 }, // ECharts async chunk is ~1.1 MB – suppress
  ...(mode === 'test' ? { resolve: { conditions: ['browser'] } } : {}),
  // ...
}));
```

Mocking Leaflet in component tests:

`ActivityMap.svelte` dynamically imports Leaflet in `onMount` . In jsdom, Leaflet's browser DOM APIs are unavailable. Mock Leaflet at the module level:

```
vi.mock('leaflet', () => { /* minimal stubs for map, tileLayer, control, etc. */ });
vi.mock('leaflet/dist/leaflet.css', () => ({}));
```

Also stub `ResizeObserver` (not implemented in jsdom):

```
vi.stubGlobal('ResizeObserver', class { observe = vi.fn(); disconnect = vi.fn(); });
```

There are no integration or end-to-end tests at present. The API routes for sync are exercised manually — see section 3.11 for local setup.

Shared test helper — `makeBaseActivity`

`src/lib/test-utils.ts` exports `makeBaseActivity(overrides?)` , a single factory for building `Activity` fixture objects in tests (PR #108):

```
import { makeBaseActivity } from '$lib/test-utils';

const activity = makeBaseActivity({ id: 'run-1', records: [/* ... */] });
```

The factory fills every required `Activity` field with safe defaults and computes `availableChannels` automatically via `channelsPresentInRecords(records)` . It also sets `anomalies: []` as the default empty anomaly list, matching `normalise()` 's initial state before `detectAnomalies()` is called. Pass `overrides.availableChannels` to supply a manual set when the test is specifically about channel visibility rather than parser behaviour.

All test files should import this helper rather than duplicating the full `Activity` literal — it prevents future interface changes from requiring edits across every test file.